

MATLAB Assignment

Atrisa
Hormozzadeh

--

11/2024

5

Elec1703

—

Dr Dragan Indjin

Table of Content

1.Functions	3
1.1 Function 1	3
1.2 Function 2	5
2. McLaurin Expansion – numerical analysis	7
2.1 Factorial function	7
2.2 McLaurin expansion of sinus function	10
3. M-files and constructing programs in MATLAB	16
Matrix types	16
Matrix transpose	17
Matrix determinant and inversion	17
Solving systems of linear equations	17

1.Functions

1.1 Function 1

-Function code:

```
function1.m × function2.m × better_plot.m × untitled4.m × untitled8.m × my_factorial.m × +
/MATLAB Drive/Examples/R2024b/matlab/FunctionWithOneOutputExample/function1.m
1 function [y1]=function1(x)
2 % this function expresses f1(x)=2*sin(x)*cos(x)/x
3 y1=2*sin(x).*(cos(x)./x) %this line of code defines our function
4 % the . factor makes sure that the multiplication is done elemental wise
5 % and prevents any mistakes caused by multiplying arrays (will discuss
6 % further in report)|
7 end
8
```

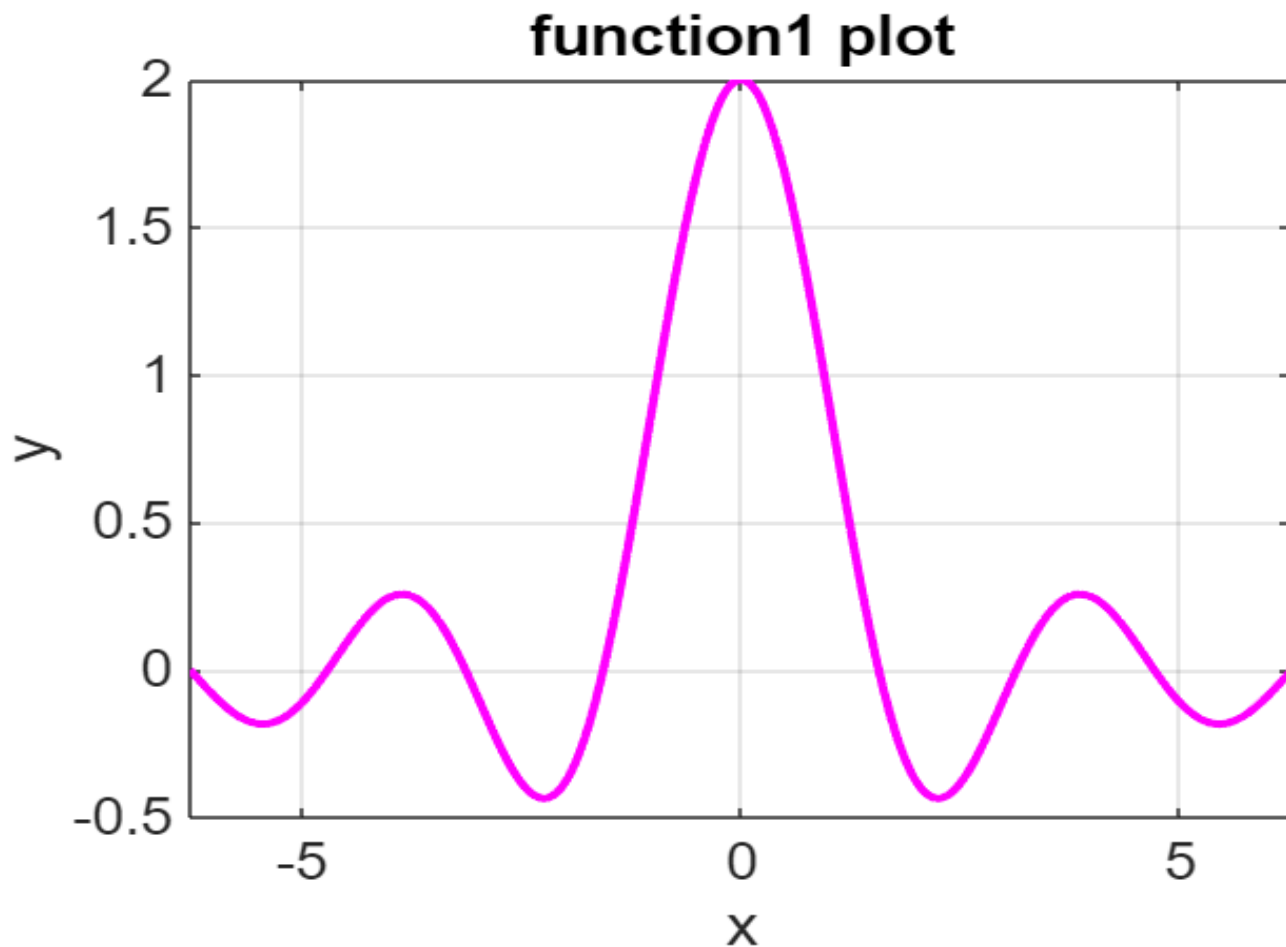
-Main code:

```
function1.m × maincode1.m × better_plot.m × function2.m × untitled4.m × untitled8.m × > v
/MATLAB Drive/maincode1.m
1 clear
2 close all
3 clc
4
5 x=linspace(-2*pi, 2*pi,1000);
6 y=function1(x)
7 better_plot(x,y,'-m','x','y')
8 title("function1 plot")
9 grid on
10 % better plot is coded according to the instructions which is in another
11 % file|
```

-“Better plot” code:

```
function1.m × maincode1.m × better_plot.m × function2.m × untitled4.m × untitled8.m × > v
/MATLAB Drive/Examples/R2024b/matlab/FunctionWithOneOutputExample/better_plot.m
1 function f=better_plot(x,y,Linespec,xlab,ylab)
2 figure
3 plot(x,y,Linespec,'LineWidth',3)
4 xlabel(xlab)
5 ylabel(ylab)
6 set(gca,'FontSize',20)
7 grid on
8 end
9 % this is the code written for plotting more efficiently on this assignment
10 % the code is written according to the instructions given|
```

-Function's plot:

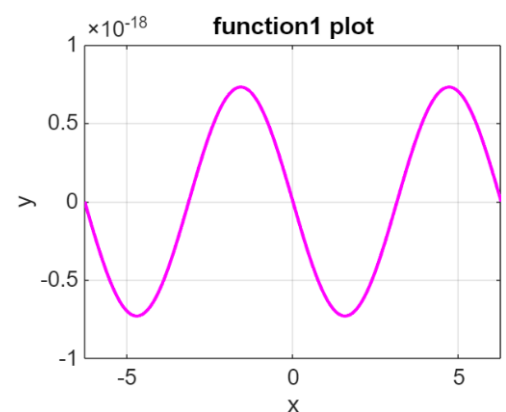


-Additional comments:

The objectives of this task were:

- Using the function factor to code any given function.
- How to plot said function and design the plot accordingly

An issue faced when writing this code was not using the '.' (dot) operator. This caused the MATLAB to not recognise the arrays and not multiply elemental wise. Even though the code would work, and no error would come up the answers were wrong and if the plot was graphed it, was a much different graph. (see below for said plot). this error was caused due to matlab performing matrix multiplication and not elemental-wise multiplication and dividing. This only shows how detailed one must be when coding in MATLAB.



1.2 Function 2

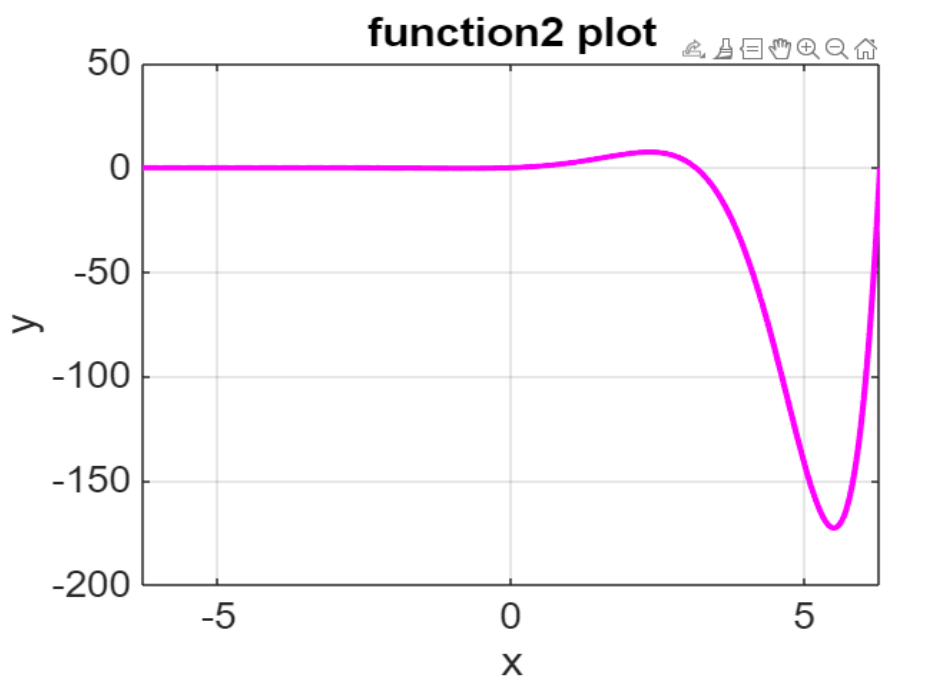
-Function code:

```
function1.m × maincode1.m × better_plot.m × function2.m × maincode2.m × my_factorial.m × +
/MATLAB Drive/Examples/R2024b/matlab/FunctionWithOneOutputExample/function2.m
1 function [y2]=function2(x)
2 %this function will define f2(x)=e^x*sin(x)
3 %e = exp(x); %this line is to show how MATLAB defines e
4 y2=exp(x).*sin(x) %this line of code defines our function
5 end
6
```

-Main code:

```
function1.m × maincode1.m × better_plot.m × function2.m × maincode2.m × my_factorial.m × +
/MATLAB Drive/maincode2.m
1 clear
2 close all
3 clc
4
5 x=linspace(-2*pi, 2*pi,1000);
6 y=function2(x)
7 better_plot(x,y,'-m','x','y')
8 title("function2 plot")
9 grid on
10 % This the main code that plots the functions
```

-Function's plot:



-Additional comments:

In this code as well, the use of ‘.’ Operator is essential. In the snapshot below, you can see the error MATLAB generates if ‘.’ Is deleted from the function code and the main code is run. As seen, the error indicates that the multiplying operation cannot be done due to the “matrixes” having different number of rows and columns. So, even though $\sin(x)$ and e^x are both numbers, MATLAB does not recognise them as such hence why we must use the dot operator. (or write a for loop code which is what the dot operator does for us)

Command Window

```
Error using *  
Incorrect dimensions for matrix multiplication. Check that the number of columns in the first matrix matches the number of rows in the second matrix. To operate on each element of the matrix individually, use TIMES (.* ) for elementwise multiplication.  
  
Error in function2 (line 4)  
y2=exp(x)*sin(x) %this line of code defines our function  
      ^  
Error in maincode2 (line 6)  
y=function2(x)  
^^^^^^^^^^^^
```

-Function test:

Here is the code for the data $x=\pi$ in function 1 ($f_1(x) = 2((\sin x) \frac{\cos x}{x})$) and how MATLAB displays the answer.

function1.m ×	maincode1.m ×	better_pl
/MATLAB Drive/practice1.m		
1	x=pi	
2	y1=function1(x)	
3	disp(y1)	

Command Window

```
>> practice1  
  
x =  
  
    3.1416  
  
y1 =  
  
-7.7963e-17
```

As seen, y1 is very close to zero when $x= \pi$ which is logical hence why our code in MATLAB is correct.

For the second function, we choose $x=1$ for $f_2(x) = e^x \sin x$ and test the code in MATLAB. By checking the answer in a calculator, we can confirm that our code is correct. (note that x is in radian).

/MATLAB Drive/practice1.m

1	x=1
2	y2=function2(x)
3	disp(y2)

>> practice1

```
x =  
  
    1  
  
y2 =  
  
    2.2874
```

Question 1 done.

2. McLaurin Expansion – numerical analysis

2.1 Factorial function

-Factorial Function code:

```
1 function [M]= my_factorial(n)
2 M=1;
3 validateattributes(n, {'numeric'}, {'positive', 'integer'});
4 %this line is to make sure n is positive and an integer number
5 for i= 2:n
6     M=M*i;
7 end
8 end
9 % we are writing a for loop which calculates the previous results and then
10 % multiplies M with the new n so if n=4, new M is old M that was calculated
11 % when n=3 multiplied by n=4 (M4=M3*4)
```

-Code comments:

This code works on a basis of a for loop. First, I used the function operator to define my function. The output is M and the input is n. To make sure n is both positive and an integer value, line 3 was written using the valid attribute's function. The for loop was written, in which M is calculated in a way that multiplies the previous M by the new value of n, as explained in the snapshot.

-Test:

To test this function, we'll write this code and try n=4.

```
function1.m × maincode1.m × better_plot.m × function2.m × maincode2.m × my_factorial.m × maincode3.m
/MATLAB Drive/maincode3.m
1 n=4; % n=170 is the maximum value MATLAB can calculate and
2 % anything larger than this number (say n=276) MATLAB would display the ans
3 % as Inf meaning infinity
4 M=my_factorial(n);
5 disp(M)
```


Command Window

```
>> maincode3
```

```
M =
```

```
24
```

```
24
```

```
>>
```

MATLAB would display M (since a semi colon wasn't added so that it would display M as well) and then calculate 4! Which is 24. As seen in the snapshot, we can say the value calculated is correct therefore the function works properly.

-Additional Comments:

As seen in the snapshot, MATLAB can only calculate the factorial function up to $n=170$. If we choose the input any value higher than 170, MATLAB would choose the answer as infinity and therefore would display Inf as the output. In the picture below you can see when $n=197$, a value larger than 170, MATLAB would no longer be able to calculate properly and would display Inf instead.

/MATLAB Drive/maincode3.m

```
1  n=197; % n=170 is the maximux value MATLAB can calcualte and
2  % anything larger than this number (say n=276) MATLAB would diplay the ans
3  % as Inf meaning infinity
4  M=my_factorial(n)
5  disp(M)
```

Command Window

```
>> maincode3
```

```
M =
```

```
Inf
```

```
Inf
```

```
>>
```

The factorial function that is built-in on MATLAB has the same limitation that cannot calculate inputs larger than 170. Looking back at our code for our function, line 3 is to make sure n is an integer positive value. If the n given to MATLAB is negative or a fraction, MATLAB would display an error indicating that n must be an integer positive value. (Example shown below)

```
1  n=-98; % n=170 is the maximux value MATLAB can calcualte and
2  % anything larger than this number (say n=276) MATLAB would diplay the ans
3  % as Inf meaning infinity
4  M=my_factorial(n)
5  disp(M)
```



```
>> maincode3
Error using my_factorial (line 3)
Expected input to be positive.

Error in maincode3 (line 4)
M=my_factorial(n)
    ^^^^^^^^^^^^^^^^^^^
```

If this line is not written, by giving a negative input or non-integer value, MATLAB would display the answer as 1, hence why we must add this line.

2.2 McLaurin expansion of sinus function

-Function code:

```
/MATLAB Drive/my_sin.m
1 function S = my_sin(x,n)
2 % x and n both are inputs and numbers
3 % k is index of summation in the series. It starts at n=0 and
4 % increases by 1 in each term.
5 % a clear version of sin(x) in terms of k and x is available in the report.
6 S=0; %this is the initial S value
7 validateattributes(n, {'numeric'}, {'positive', 'integer'});
8 % this line is to validate the input. |
9 for k=0:n
10     S=S+((-1).^k)*((x).^(2*k+1))./my_factorial(2*k+1);
11     % the first S is added due to the for loop summing all the previous S
12     % value and then added the new value. For example, S(10)=S(1:9)+S(10).
13 end
14 end
15
```

-Code comments:

The function works using the general form of the McLaurin which is:

$$\sin(x) = x + \sum_{k=1}^n (-1)^k \frac{x^{2k+1}}{(2k+1)!}$$

In which k is a positive integer number and n is the number of terms. Both n and x are values given by the user. Line 6 to 9 is the for loop written that will calculate sin(x) according to the x and n assigned by the user.

-Test:

To test the code, n was given a value of 10 while x was chosen to be 2 (in radian). The code is shown below.

```
/MATLAB Drive/my_test_code4.m
1 n=40;
2 x=7979;
3 % for this function, x = 6392 is the threshold value that limits this
4 % function. when given a value larger than 6392, S is a very large number
5 % thus why matlab would display Inf.
6
7 % for x > 7979, MATLAB no longer consider the value a number hence why it
8 % would display NaN (Not a Number). This means that the value of S is so
9 % large that MATLAB cannot comprehend it.
10 S=my_sin(x,n);
11 disp(S)
```

The result is shown below under the name maincode4. The value of sin (2) is also calculated using MATLAB built-in operator to compare the values.

```
>> maincode4
    0.9093

>> sin(2)

ans =

    0.9093
```

-Additional Comments:

As written in the figure above, this function has a limitation in the largest S it can calculate. For x's greater than 6392, the value of S is displayed as Inf since it is very large. However, for any x greater than 7979 the value of S gets so large that MATLAB no longer recognises the value as a number and would display NaN. Outputs for 2 random x's (one is between 6392 and 7979, another is larger than 7979) are shown below.

```
>> maincode4
    Inf

>> maincode4
    NaN

>>
```

In this function x is a single real number while n is a positive integer. In the coding line 8 was written to ensure these conditions for number of terms (n). if in the main code, n is given any other value MATLAB would display an error indicating that n must be a positive integer value. (The error displayed is shown below).

```
>> maincode5
Error using my_sin (line 7)
Expected input to be integer-valued.

Error in maincode5 (line 3)
numerator1=abs(my_sin(x,n) - sin(x));
          ^^^^^^^^^^^^^
```

2.3 McLaurin expansion of sinus function- analysis

-Code:

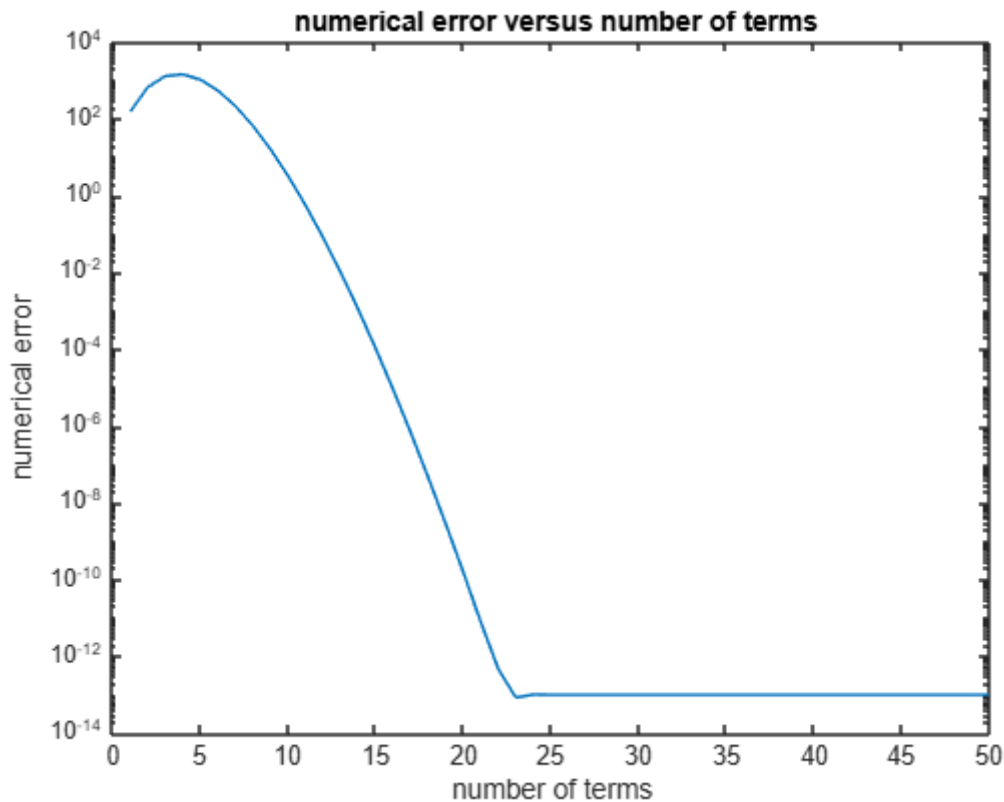
This is the code written for part a where it calculates the numerical error by changing number of terms with a fixed value of x. line 7 to 9 is for the plot.

```
/MATLAB Drive/maincode7.m
1  x = 10;
2  for n = 1:1:50
3      numeror(n)=abs(my_sin(x,n) - sin(x));
4  end
5  k = 1:50;
6
7  figure;
8  semilogy(k , numeror)
9  title(" numerical error versus number of terms")
10 xlabel("number of terms")
11 ylabel("numerical error")
```

The next snapshot is the code for part b where we calculate numerical error versus values of x over fixed values of n.

```
1  n = 10;
2  x = linspace(-10, 10, 1000);
3  numeror1=abs(my_sin(x,n) - sin(x));
4
5  n = 15;
6  x = linspace(-10, 10, 1000);
7  numeror2=abs(my_sin(x,n) - sin(x));
8
9  n = 20;
10 x = linspace(-10, 10, 1000);
11 numeror3=abs(my_sin(x,n) - sin(x));
12
13 figure;
14 x = linspace(-10, 10, 1000);
15 semilogy(x, numeror1, x, numeror2,x, numeror3);
16 title(" numerical error versus x values ")
17 legend("numeror1", "numeror2", "numeror3")
18 xlabel("x")
19 ylabel("numerical error")
```

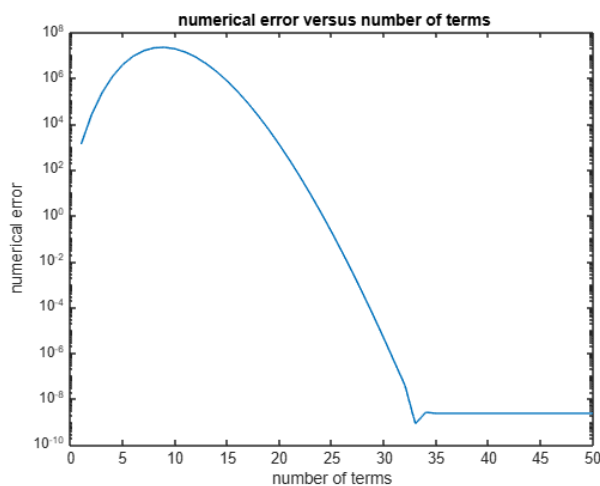
-Plot of numerical error versus number of terms for fixed value of x :



-Comments:

As seen in this plot, for larger number of terms the numerical error occurred is smaller and smaller and for n larger than 25 it can be neglected since it is very small. If we increase the value of x , we will observe that for less number of terms the numerical error is very small.

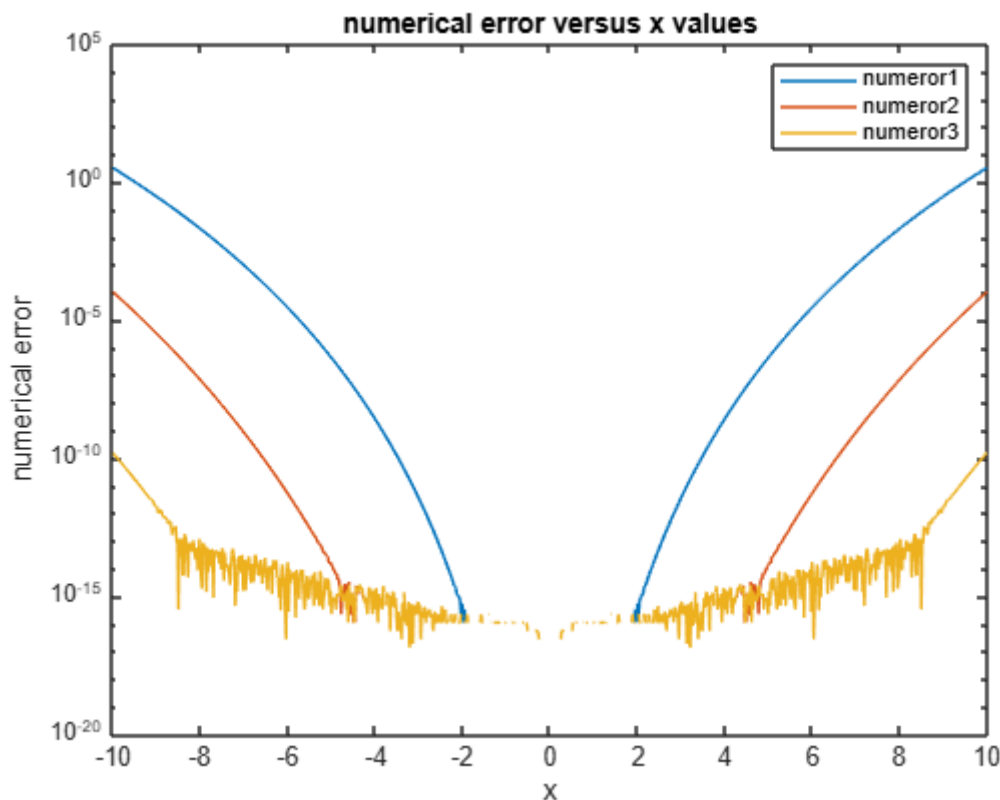
For example, if $x=20$ then for any $n > 35$ the numerical error is neglectable. (As shown in the plot below)



In this plot, $x=20$ and n is from 1 to 50.

However, if for the same value of x we choose lesser number of terms, our numerical error is larger. The reason behind this is that the larger value of n that we have, the more our McLaurin expansion is hence why the error is neglectable. Similarly, if we increase the value of x , we will need more number of terms to be precise.

-Plot of numerical error versus x for fixed value of number of terms:



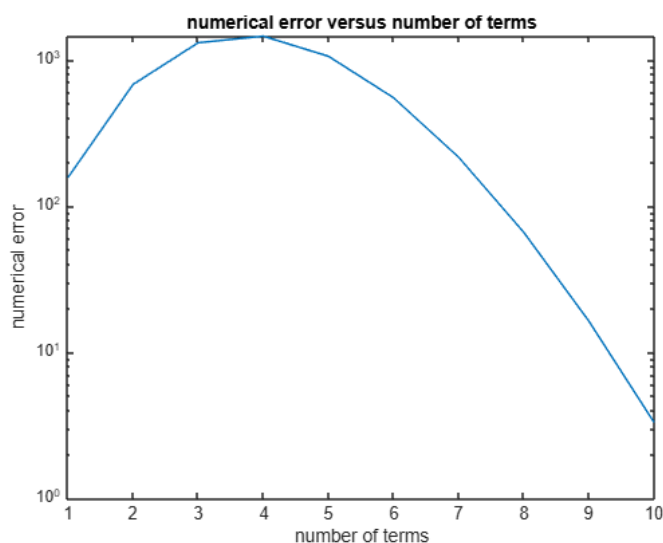
-Comments:

As seen in the plot, for lesser values of x the numerical error goes to very small values. As said before for increased values of x we will need more number of terms to be more accurate. This can be proved by looking at the difference between the plots graphed. For numerator1 which has less number of terms, the biggest error is near 10 which is a very big error while for numerator 3 which has 25 number of terms this number is significantly lower, as low as 10^{-10} , a very small number.

This observation proves that for more accuracy in Mclaurin expansion, we must push for number of terms and adjust them according to the value of x.

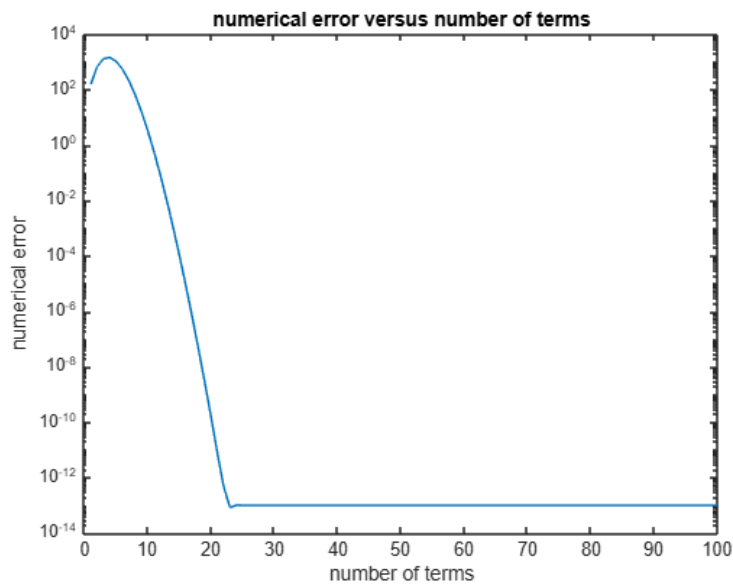
-Additional plots:

In this plot, x is 10 and n is from 1 to 10. As expected, the numerical error is much larger. (explained above)



graph for numerical error versus number of terms.

For the next plot, x is still 10 but now n is a range from 1 to 100. As seen below, from almost 25 number of terms the numerical error is a very small constant and therefore neglectable.



graph for numerical error versus number of terms.

-Additional comments:

Due to the restriction on n being a positive integer, to write the code for part b, several n was used and then using the hold on and off operator one figure plotted 3 different graphs.

Question 2 done.

3. M-files and constructing programs in MATLAB

Question	code	output	comment
Matrix types			
3.1	<code>v = [2 3 4 5 6]; diag(v)</code>	<pre>ans = 2 0 0 0 0 0 3 0 0 0 0 0 4 0 0 0 0 0 5 0 0 0 0 0 6</pre>	
3.2	<code>I = eye(3)</code>	<pre>I = 1 0 0 0 1 0 0 0 1</pre>	
3.3	<code>X = zeros(3,2)</code>	<pre>X = 0 0 0 0 0 0</pre>	
3.4	<code>Y = ones(2,4)</code>	<pre>Y = 1 1 1 1 1 1 1 1</pre>	

(A snapshot of the code)

1	<code>v = [2 3 4 5 6];</code>
2	<code>diag(v);</code>
3	<code>I = eye(3);</code>
4	<code>X = zeros(3,2);</code>
5	<code>Y = ones(2,4);</code>
6	

question	code	output
Matrix transpose		
3.5	<pre>A = [4 5 6; 11 9 1; 8 7 2]; Transpose_A = A.'; result = A .*(Transpose_A);</pre>	<pre>Transpose_A = 4 11 8 5 9 7 6 1 2 result = 16 55 48 55 81 7 48 7 4 --</pre>
Matrix determinant and inversion		
3.6	<pre>Determinant_A = det(A) C = inv(A)</pre> Comment: Since determinant of A is not zero, then A is not singular.	<pre>Determinant_A = 4.0000 C = 2.7500 8.0000 -12.2500 -3.5000 -10.0000 15.5000 1.2500 3.0000 -4.7500</pre>
Solving systems of linear equations		
3.7	<pre>B = [8;15;12]; %define B matrix % we know that x = inverse_A * B so we write the next line of code to be: x = inv(A)*B; %calculate x array % y = A\B; % to check the results we multiply A by x to see if we get B Results= A * x</pre>	<pre>X1= -5 X2= 8 X3= -2</pre>

-Code and outputs snapshots:

```
1 A = [4 5 6; 11 9 1; 8 7 2]; % Define A matrix
2 Transpose_A = A.';
3 result = A.*(Transpose_A);
4 Determinant_A = det(A) %A is therefore not singular
5 C = inv(A);
6 B = [8;15;12]; %define B matrix
7 % we know that x = inverse_A * B so we write the next line of code to be:
8 x = inv(A)*B %calculate x array
9 % y = A\B;
10 % to check the results we multiply A by x to see if we get B
11 Results= A * x;
```

Determinant_A =

4.0000

C =

2.7500	8.0000	-12.2500
-3.5000	-10.0000	15.5000
1.2500	3.0000	-4.7500

x =

-5.0000
8.0000
-2.0000

Results =

8.0000
15.0000
12.0000

Question 3 done.